

Executive Summary

Recent advances in aligning large language models (LLMs) to user preferences have employed various RL-based fine-tuning methods. Key approaches include traditional RLHF/PPO (OpenAI’s InstructGPT), Direct Preference Optimization (DPO) without explicit RL, and newer variants like Group Relative PPO (GRPO) and Reinforcement Learning from AI Feedback (RLAIF). We systematically compare the training configurations of these methods for ~7B-parameter models. For each method, we tabulate model size, optimizer and learning-rate details, batch and sequence sizes, training duration, compute, fine-tuning strategy (full vs LoRA), KL or entropy regularization, reward-model setup, and other tricks. Our analysis reveals common patterns (e.g. AdamW or RMSprop with LR=1e-6, cosine schedules, minibatch RL updates) as well as divergences (such as GRPO’s group-based advantage vs PPO’s critic, or DPO’s avoidance of RL entirely). We synthesize best-practice hyperparameters for a 7B model under a GRPO-style pipeline (with options for LoRA vs full tuning), practical measures to prevent overfitting (e.g. mixing in pretraining gradients, early stopping, dropout), and propose ablations (e.g. effect of KL penalty, advantage normalization, EMA). Figures and mermaid diagrams illustrate a generic training pipeline and timeline of phases. The comparative table below and detailed descriptions (with citations) capture the hyperparameter and implementation choices in seminal papers [23†L2428-L2434] [55†L1067-L1073] .

RLHF (PPO) – InstructGPT (Ouyang et al., 2022)

- **Model & Setup:** Fine-tuned GPT-3 variants (1.3B and 6B models) using Proximal Policy Optimization (PPO) [23†L2428-L2434] . Supervised fine-tuning (SFT) first for 16 epochs (dropout 0.2, cosine LR decay to 10% final, *no* warmup). SFT LR=9.65×10⁻⁶ for 1.3B/6B (batch size 32) [23†L2428-L2434] . Reward model (RM): a single 6B model trained 1 epoch on human-ranked comparisons (LR=9×10⁻⁶, cos decay, batch 64) [23†L2428-L2434] .
- **Optimizer:** AdamW (betas $\beta_1 \approx 0.9$, $\beta_2 \approx 0.95$, WD=0.1) for all stages [23†L2428-L2434] .
- **RL Stage (PPO):** 256K episodes (~31K unique prompts) [23†L2479-L2488] . Each iteration: batch size 512 tokens (split into 8 minibatches of 64) [23†L2479-L2488] , one gradient epoch over each batch. Constant LR with 10-iteration linear warmup (starting at 1/10 peak) [23†L2479-L2488] . Peak LR tuned per model (found ~9e-6 for 6B policy; RM and value nets used 9e-6 for 1.3/6B, 5e-6 for 175B) [23†L2493-L2498] . PPO clip $\epsilon=0.2$, sampling temperature 1 [23†L2479-L2488] . No reward discount ($\gamma=1$) and no explicit entropy bonus. A per-token KL penalty (from the SFT model) was added to the reward [61†L569-L576] to prevent divergence. The value network was a 6B model (initialized from the reward model) [23†L2493-L2498] .
- **Fine-tuning:** Full-model (no LoRA) updates. Pretraining mix: at each RL step, gradients from the original pretraining data were blended in (8× more pretraining data than RL data, with a tunable coefficient) to prevent drift [23†L2500-L2508] . EMA of policy weights applied (decay 0.992) during RL [23†L2479-L2488] .
- **Overfitting & Regularization:** Dropout 0.2 in SFT. Mixed pretraining gradients and KL penalty helped prevent collapse. Early stopping based on reward-model scores was used for SFT baseline selection. Checkpointing: best models selected by validation reward model score. RLHF training was monitored on public NLP benchmarks to detect regressions and was ended after 256K episodes (empirically sufficient) [23†L2479-L2488] .

- **Compute:** Exact GPU-days not reported, but RLHF required significant compute (training 175B policy with 256K updates). (InstructGPT used large-scale compute; see discussion in text).

DPO – Direct Preference Optimization (Rafailov et al., 2023)

- **Model & Setup:** Fine-tuned open LMs (up to ~6B parameters [56†L102-L106] , e.g. GPT2/GPT-J/GPT-NeoX family) directly on preference comparisons. No explicit reward model or RL steps; instead a single-stage loss. For example, in summarization/dialog tasks they fine-tune a policy with binary preference labels.
- **Optimizer & LR:** Used RMSprop (rather than Adam) with learning rate 1×10^{-6} (linear warmup from 0 over 150 steps) [41†L1170-L1173] . Batch size 64 examples. For certain tasks (TL;DR summarization) they adjust batch or MLP layers, but basic hyperparameters are similar [41†L1170-L1173] .
- **Training Data:** Prefers pairs of outputs (preferred vs dispreferred) per prompt. They often pre-generate multiple completions (e.g. 4 outputs per prompt) using an SFT model, then label with a ground-truth classifier or human labeler, and form preference pairs.
- **Loss:** A pairwise logit-based loss: $\text{maximize } \beta * (\text{logit}(\text{pref}) - \text{logit}(\text{anti}))$ with a learned or fixed scaling β [41†L1165-L1173] . This effectively increases the probability gap between preferred and dispreferred completions. β hyperparameter controls KL-like penalty implicitly (set via eqn. above). No separate KL term; the formulation already biases the policy.
- **Fine-tuning:** Full-model updates (no LoRA in reported experiments). Usually initialized from an SFT model. No separate value network, advantage, or entropy bonus – it's like supervised fine-tuning on preferences.
- **Sequence & Batch:** Sequences up to 1024 tokens typically, mini-batches of 64 preference pairs. No mention of gradient clipping or dropout (likely defaults).
- **Regularization:** They simply rely on early stopping and validation. They note training is stable (unlike RL), so overfitting less pronounced. No data augmentation or dropout reported.
- **Compute:** Moderate – training usually takes a few epochs over tens of thousands of preference examples. (E.g. sentiment example: 25K prefixes, generate 4 completions each, 6 comparisons each – training a 350M or 6B model).
- **Source:** DPO paper (NeurIPS 2023) [41†L1170-L1173] .

GRPO – Group Relative PPO (Peng et al., 2024)

- **Paper & Model:** DeepSeekMath “RL” with a 7B model [55†L1079-L1087] . Builds on a specially-pretrained 7B (DeepSeekMath-Base) for math reasoning, then fine-tunes with GRPO (an actor-only variant of PPO).
- **Data:** RL data consisted only of chain-of-thought math problems (e.g. GSM8K, MATH benchmarks) – about 144K questions [55†L1058-L1066] . Reward labels came from a fixed 7B reward model (trained earlier).
- **Optimizer & LR:** AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $WD=0.1$ as in pretraining) [55†L1067-L1073] . Policy LR = 1×10^{-6} , Reward-model LR = 2×10^{-5} [55†L1067-L1073] . Multi-step schedule not detailed (likely constant or slight decay).
- **Batch & Sequence:** For each prompt, $G=64$ outputs are sampled from the current policy [55†L1067-L1073] . Batch size = 1024 tokens (i.e. $\sim 1024/1024=1$ prompt per batch, or group of 64?). Max sequence length = 1024 tokens for generations [55†L1067-L1073] . The formulation implies generating 64 completions at once, scoring them, then performing 1 gradient update per prompt (“single update per exploration stage”).

- **GRPO Specifics:** No critic; baseline is the **group mean reward**. Advantage for each completion is computed as $(r_i - \text{mean}(r))/\text{std}(r)$ (normalizing by the group's std) [51†L205-L213] . A KL divergence term (coefficient 0.04) is added to the loss to keep policy close to the original SFT policy [55†L1067-L1073] . No explicit entropy bonus was reported.
- **Fine-tuning:** Full-model fine-tune (no LoRA). The reference policy for KL was the initial SFT model. After each batch, GRPO updates the policy once. Reward model was occasionally retrained on new data (iterative loop) using a 10% replay buffer of past data.
- **Regularization:** KL penalty 0.04 [55†L1067-L1073] and advantage normalization inherently stabilize training. Dropout (from SFT) likely still active (0.2). No other data augmentation or distillation reported.
- **Compute:** Training was made memory-efficient by using GRPO. They report completing 256k prompts of RL (31k unique) with one update each, which is much less than PPO's multi-epoch approach. Specific GPU-days not given, but thanks to single-update-per-batch, GRPO is more efficient than PPO for this setting [55†L1067-L1073] .
- **Source:** DeepSeekMath-Base & -RL (Peng et al. 2024) [55†L1067-L1073] . (Note: HF TRL docs also illustrate GRPO's objective [51†L231-L239] .)

【53†embed_image】 *Figure:* GRPO training schematic: for each prompt, the policy generates a group of responses which are scored by the reward model; advantages are computed relative to the group's mean (normalized by its std) [51†L205-L213] , and the policy is updated with a KL penalty to the original policy.

RLAIF / HRLAIF – (Li et al., 2024)

- **Model & Setup:** Llama2-13B-base (bilingual) fine-tuned for conversation. Trained with “reinforcement learning from AI feedback”: using ChatGPT as the labeler instead of humans [34†L13-L22] .
- **Reward Model:** Same 13B architecture (Llama-13B). Trained with Cosine schedule, 4 epochs, LR $\text{max}=2 \times 10^{-6}$, batch = 1 prompt with 5 ChatGPT responses (effectively batch size 16 prompts across 16 GPUs) [36†L448-L457] . Sequence length 4096.
- **RL Stage (PPO):** Batch = 1 prompt per GPU (16 GPUs, no minibatching), sequence length 4096 [36†L456-L464] . Cosine LR schedule, max LR $=3 \times 10^{-7}$ (no warmup specified). PPO updates interleaved with ChatGPT feedback. Likely default clip (0.2) and value network (13B) but not explicitly given. No mention of KL term – but presumably small or none since they optimized directly with ChatGPT scores.
- **Fine-tuning:** Full-model (no LoRA) updates. Reward model and policy are same size (13B).
- **Regularization:** No explicit entropy bonus or advantage normalization discussed. To prevent overfitting, they limited training (~360 steps) and introduced “Hybrid” labeling (HRLAIF) with improved AI label quality on hard categories [34†L23-L31] [36†L462-L470] . Evaluations every 120 steps to monitor degradation [36†L508-L517] .
- **Overfitting:** They observed that after ~360 PPO steps, model helpfulness sharply declined due to overfitting the AI-derived reward [36†L515-L523] . HRLAIF improved label accuracy to mitigate this. They also added a small set of harmful prompts with AI red-teaming to further regularize. No dropout or explicit early stopping reported aside from stepping out early after tens of thousands of tokens.
- **Compute:** Training used 16×40GB A100 GPUs for PPO. They report ~125 seconds per training step on 13B [36†L470-L478] . Total RL was on the order of a few hundred steps (each step = batch of 16).

• **Sources:** HRLAIF paper (Tencent, 2024) [36†L448-L457] [36†L515-L523] . (Basic RLAIIF described qualitatively in Bai et al. 2022b; see Li et al. for concrete setup.)

Comparative Hyperparameters and Design Choices (7B-scale models)

Method (Paper)	Model Size (params)	Fine-tune (LoRA)	Optimizer (β_1, β_2, WD)	LR & Schedule (peak, warmup)	Batch & Seq Len	Steps/Epochs	RL Specific (KL, entropy adv norm, critic)
InstructGPT (Ouyang 2022) [23†L2428- L2434] [23†L2479- L2488]	1.3B, 6B (we use 6B)	Full (no LoRA)	AdamW (0.9, 0.95, 0.1) [23†L2428- L2434]	SFT: LR~1e-5 (cos decay to 0), <i>no</i> warmup [23†L2428- L2434] ; RL: LR tuned (~9e-6 constant, with 10-iter warmup) [23†L2479- L2488]	SFT: 32 (1.3B/6B), 8 (175B); RL: 512 tokens/ it (8×64) [23†L2479- L2488] ; seq=2000 (ctx 2k, response ≤1k) [23†L2424- L2432]	SFT: 16 epochs [23†L2428- L2434] ; RM: 1 epoch; RL: 256K episodes (~31K unique prompts) [23†L2479- L2488]	PPO (no entropy bo clip=0.2; $\gamma=$ Adv norm: none (generalize adv) [23†L2479- L2488] ; KL per-token penalty fro reference (coefficient tunable, 0.0 0.04 in eqn [61†L569- L576]); Cr 6B value network (initialized RM) [23†L2493- L2498] ; EL yes (decay 0.992) [23†L2479- L2488]

Method (Paper)	Model Size (params)	Fine-tune (LoRA)	Optimizer (β_1, β_2, WD)	LR & Schedule (peak, warmup)	Batch & Seq Len	Steps/Epochs	RL Specifics (KL, entropy, adv norm, critic)
DPO (Rafailov 2023) 【56†L102-L106】 【41†L1170-L1173】	6B (max in paper)	Full (no LoRA)	RMSprop (no β 's given)	LR=1e-6, warmup 150 steps 【41†L1170-L1173】 (constant schedule used)	Batch=64 prefs per step 【41†L1170-L1173】 ; seq $\approx$ 1024	~few epochs on pref. data (ex: sentiment: 3 epochs, summarization: a few)	Objective: separate RL from binary XE loss on preferred/dispreferred pairs 【41†L1165-L1173】 . Implied β parameter loss (like KL scale). No explicit KL penalty term, entropy bonus. No separate critic. Advantage entropy not applicable.
GRPO (Peng 2024) 【55†L1067-L1073】 【51†L205-L213】	7B (DeepSeekMath-RL)	Full (no LoRA)	AdamW (0.9, 0.95, 0.1) 【55†L1067-L1073】	Policy LR=1e-6; RM LR=2e-5 【55†L1067-L1073】 (cosine/constant not specified, small overall)	Batch: 1024 tokens (effectively 1 prompt per batch) 【55†L1067-L1073】 ; Sample 64 completions per prompt 【55†L1067-L1073】 ; seq=1024	RL: "online" (no fixed episode count). ~144K prompts total; one update per prompt group 【55†L1067-L1073】 .	GRPO: Advantage = $(r - \text{mean}(r)) / \text{std}(r)$ 【51†L205-L213】 . KL penalty = 0. (via loss term) 【55†L1067-L1073】 . No entropy bonus mentioned. Critic: none (baseline = group mean). EMA: none mentioned (likely not used).

Method (Paper)	Model Size (params)	Fine-tune (LoRA)	Optimizer (β_1, β_2, WD)	LR & Schedule (peak, warmup)	Batch & Seq Len	Steps/Epochs	RL Specific (KL, entropy, adv norm, critic)
RLAIF/ HRLAIF (Li 2024) 【36†L448-L457】 【36†L515-L523】	13B (Llama2)	Full (no LoRA)	(not explicitly stated, likely AdamW)	RM LR max=2e-6 (cosine, 4 epochs) 【36†L448-L457】 ; PPO LR max=3e-7 (cosine, no warmup) 【36†L456-L464】	RM batch=1 prompt×5 responses/ GPU (16 GPUs total) 【36†L448-L457】 ; PPO batch=1 prompt/ GPU (16 GPUs) 【36†L456-L464】 ; seq=4096	PPO: a few hundred steps (e.g. 360) 【36†L516-L523】 ; RM: ~4 epochs	PPO w/ ChatGPT rewards. KL Entropy : Not specified; assumed default PPO clipping. No advantage normalization mention. C likely 13B v model. EM not reported

(Values approximated or assumed when unspecified in papers.)

Common Patterns and Key Differences

- **Optimizers & Learning Rates:** Most methods use Adam-based optimizers (AdamW or RMSprop) with **very low learning rates** ($\sim 1e-6$). PPO/RL methods often apply cosine decay to a small fraction ($\approx 10\%$) of the initial LR, with short warmup (if any) 【23†L2428-L2434】 【36†L448-L457】. SFT stages often use slightly higher LR ($\sim 1e-5$) with cosine schedules 【23†L2428-L2434】.
- **Batch and Sequence Sizes:** Due to large model size, batch sizes are small (per-GPU $\approx 1-8$ prompts; effective tokens per batch $\sim 32-512$). Sequence lengths range 1024–4096 depending on context length needs 【55†L1067-L1073】 【36†L448-L457】. RLHF papers often use many “minibatches” (e.g. 8×64 in InstructGPT 【23†L2479-L2488】). GRPO uses one prompt per batch but generates 64 completions.
- **Training Duration:** SFT usually a handful of epochs on labeled data. Reward model training is short (1–4 epochs). RL stages vary: RLHF used 256K episodes in InstructGPT 【23†L2479-L2488】; GRPO used $\sim 144K$ prompts (one update each); HRLAIF used only a few hundred PPO steps. DPO fine-tuning is only as long as supervised fine-tuning (no RL).
- **Fine-tuning Style:** All examined papers fine-tune the *full model weights*. No method here used LoRA or adapter modules on the 7B-scale (though LoRA is an option in libraries 【51†L125-L134】). We

note that LoRA could greatly reduce compute, but none of these academic papers reported it. (In practice, some open-source projects have since experimented with LoRA for PPO.)

- **KL Penalty:** RLHF/PPO approaches include KL penalties (often implemented as per-token KL to reference policy [61†L569-L576]) to prevent the policy drifting. InstructGPT tuned a KL coefficient (~0.015–0.04) in PPO [61†L569-L576] . GRPO explicitly adds KL term 0.04 [55†L1067-L1073] . DPO’s update inherently enforces a similar constraint via the implicit β scaling in its loss [41†L1170-L1173] . HRLAIF did not emphasize KL (likely relied on small LR).
- **Reward Models and Feedback:** RLHF/PPO uses a separately trained reward model (6B for InstructGPT, 13B for HRLAIF) to score outputs [23†L2428-L2434] [36†L448-L457] . DPO and some RLAIIF variants bypass training a reward model by using either a pre-existing classifier or LLM (ChatGPT) as an annotator [41†L1165-L1173] [34†L13-L22] . GRPO still uses a reward model but re-trains it iteratively with a replay buffer [55†L1067-L1073] .
- **Baseline/Critic:** PPO uses a value network (critic) to estimate the baseline. InstructGPT initializes the critic from the reward model [23†L2493-L2498] . In contrast, GRPO *forgoes any value network* – the baseline is the group-average reward [51†L205-L213] . DPO and basic RLAIIF simply optimize directly without advantage estimation.
- **Advantage Normalization:** PPO methods (InstructGPT, HRLAIF) use generalized advantage estimation (GAE) with no discount. InstructGPT did not explicitly normalize advantages beyond GAE. GRPO introduces a *group-wise advantage normalization*: each answer’s advantage is (its reward – mean) normalized by the group standard deviation [51†L205-L213] . HRLAIF/PPO did not report any special scaling of advantages.
- **Entropy Bonus:** None of the surveyed papers highlighted using an explicit entropy regularization term. PPO often has it by default (but was not emphasized). DPO has no notion of entropy.
- **Overfitting Prevention:** Common tactics include **dropout** in SFT (0.2 used in InstructGPT) [23†L2428-L2434] , **data mixing** (pretrain gradients mixed into PPO to retain general skills [23†L2500-L2508]), and **KL penalties**. InstructGPT stopped RM training after 1 epoch to avoid overfitting [23†L2420-L2427] . RLHF training lengths were chosen to maximize evaluation reward without overflow. HRLAIF noted overfitting (helpfulness collapse) and halted training early (mid-checkpoint used) [36†L515-L523] . These approaches suggest **monitoring** on held-out tasks and early stopping are crucial. None of the works used data augmentation or adversarial augmentation except HRLAIF’s AI red-teaming prompts.
- **Evaluation Frequency:** PPO-based works evaluated periodically using the reward model or fixed benchmarks (e.g. every few hundred steps in HRLAIF [36†L508-L517] , or by human eval at the end). GRPO evaluated only after full RL runs on reasoning benchmarks [55†L1079-L1087] . Frequent evaluation (every few steps) is not detailed for GRPO but was done in HRLAIF. Reward-model checkpoints were picked by validation reward score (InstructGPT).
- **Compute:** RL fine-tuning is expensive. InstructGPT 175B ran on dozens of GPUs/TPUs; 6B policy needed less. GRPO claims efficiency gains (only 1 update per group reduces compute). HRLAIF used 8×A100s at 125s/step for 13B. DPO’s training (no RL) was relatively lightweight (few GPU-days for ~6B model). We summarize key resource hints in the table above.

Recommended Baseline Hyperparameters (7B Model, GRPO-style)

For experimenting with GRPO-like RL on a 7B language model (or 6–8B range), a reasonable baseline configuration is:

- **Optimizer:** AdamW with $\beta_1=0.9$, $\beta_2=0.95$, weight decay ≈ 0.1 (following DeepSeek and InstructGPT) [55†L1067-L1073] [23†L2428-L2434] .
- **Learning Rate:** Start with 1×10^{-6} for the policy (as in Peng et al.) [55†L1067-L1073] . If using a reward model that needs fine-tuning, use $\sim 2 \times 10^{-5}$. Use a cosine or constant schedule; if warmup used, $\sim 0.1 \times$ peak for a few iterations. You can sweep LR around this order of magnitude, but keep it low (e.g. [5e-7, 2e-6]).
- **Batch and Seq Length:** Prompt+completion lengths = 1024 tokens (or what your problem needs). For each prompt, generate $G=64$ completions. Batch size ≈ 1024 tokens per gradient step (this could be e.g. 1 prompt $\times$ 64 completions of ~ 16 tokens each). If memory limited, reduce group size or use gradient accumulation.
- **KL Penalty:** Include a KL-divergence term to the original model in the loss. A coefficient around **0.01–0.1** can be tuned; Peng et al. used 0.04 [55†L1067-L1073] . This term prevents collapse.
- **Advantage Computation:** Use GRPO’s group-relative advantages: for each group of completions, compute mean and std of rewards; advantage = $(r_i - \text{mean})/\text{std}$ [51†L205-L213] . This normalization stabilizes learning. (If you want ablation, try turning off std-scaling to see effect.)
- **Fine-tuning vs LoRA:** For 7B, full-model updates may fit on a high-memory GPU cluster (e.g. 8+ A100s). However, LoRA (rank e.g. 16–32) can greatly reduce memory and ops. If you use LoRA (via PEFT), target attention and feedforward layers, same LR but possibly a slightly higher peak (e.g. $2 \times$). No official LoRA experiments exist here, so treat it as an open question – our recommendation is full fine-tuning for fidelity, with LoRA as an efficiency variant.
- **Reward Model:** Use a similarly-sized reward model (e.g. 7B) if possible for consistency, or smaller (if compute-limited). Train it on comparisons first (e.g. 1 epoch, LR $\sim 1\text{e-}5$ – $2\text{e-}5$, cos decay) [55†L1067-L1073] . Always freeze or slowly update the reward model during policy RL.
- **Other PPO Settings:** Clip $\epsilon \approx 0.1$ – 0.2 . No reward discount ($\gamma=1$) unless tasks are sequential. Batch all collected completions before update (i.e. one epoch on the group).
- **Regularization:** Keep any dropout from SFT (e.g. 0.1–0.2). Mix in some supervised data occasionally to prevent drifting (like mixing 1:1 pretraining:RL gradients [23†L2500-L2508]). Early stopping based on reward-model validation or hold-out tasks. Use EMA of policy weights if you see instability ($\beta \approx 0.995$).
- **Checkpoint/Eval:** Save and evaluate model every few hundred or thousand updates on a validation set (with the reward model and, if possible, human proxies). Stop when metrics (helpfulness, accuracy) saturate or start to decline.

Overfitting Prevention and Monitoring

Based on the literature, we recommend the following strategies:

- **Monitor with Validation Rewards and Benchmarks:** Track progress on held-out prompts and tasks (e.g. math questions, benchmarks like GSM8K) every N steps. Sudden drops (as seen in HRLAIF after ~ 360 steps [36†L515-L523]) indicate overfitting. Use these metrics to decide when to stop training.

- **Use KL Regularization:** A small KL penalty (0.01–0.1) is crucial to prevent the model from drifting too far. Tune this coefficient. Increasing KL weight often prevents runaway specialization.
- **Mix in Pretraining or SFT Objectives:** Interleave PPO updates with some original LM or SFT training on high-quality instructions. InstructGPT found mixing ~1:8 (supervised:RL) helped retain generality [23†L2500-L2508] .
- **Dropout and Weight Decay:** Retain dropout used in original model (e.g. 0.1–0.2). Keep a small weight decay (0.01–0.1) to discourage parameter blow-up.
- **Limit RL Steps:** The alignment reward landscape may overfit with too much training. For a 7B model, a few hundred thousand tokens of RL might suffice; beyond that check for degradation. In HRLAIF, the best checkpoint was early in training [36†L515-L523] .
- **Gradient Clipping:** To stabilize training, clip gradients (e.g. by norm 1.0) if you observe spikes.
- **Ensemble Averaging (EMA):** Maintain an EMA of model weights (decay ~0.99) and optionally evaluate using the EMA model [23†L2479-L2488] , which can be more stable than the instant weights.
- **Reward Model Calibration:** Ensure the reward model does not assign extreme scores. Apply reward rescaling or bias as in InstructGPT (normalized so demos have mean 0) [61†L569-L576] .
- **Regular Evaluation by Humans or Strong LLMs:** Periodically have human raters or a stronger model (GPT-4/ChatGPT) judge outputs to catch alignment drift not captured by the automated RM.

Suggested Ablation Experiments

To understand the impact of key design choices, we suggest these ablations:

- **KL Penalty Strength:** Vary the KL coefficient (e.g. 0 vs 0.01 vs 0.1) to see how tightly the policy stays near the reference. GRPO vs PPO formulations for advantages may respond differently to this.
- **Advantage Normalization:** For GRPO, compare using group-wise $(r - \text{mean}) / \text{std}$ (default) vs no normalization vs using global batch normalization [51†L205-L213] . Observe effects on training stability and final quality.
- **Entropy Regularization:** Try adding an explicit entropy bonus ($\beta_{\text{entropy}}=0.01-0.1$) versus none, to encourage exploration and avoid premature convergence.
- **EMA vs No EMA:** Train with and without weight-averaging (EMA) during RL, to quantify its smoothing effect on final performance (InstructGPT used EMA [23†L2479-L2488]).
- **Full vs LoRA Fine-Tuning:** If resources allow, run experiments with full-tuning and with a LoRA adapter (e.g. rank 16) to compare convergence speed, peak performance, and susceptibility to overfitting.
- **Batch Size / Group Size:** For GRPO, vary the group size G (e.g. 16, 32, 64, 128) to see how many samples per prompt are optimal. Larger G gives better baseline estimates but costs more sampling.
- **Replay Buffer Use:** In GRPO, test with and without the 10% replay buffer for RM training, to see if updating the RM online is beneficial.
- **Mixing Supervised Data:** In PPO, ablate mixing in pretraining data (as in “pretrain gradient” trick) vs plain PPO, to measure effect on out-of-domain performance.
- **Different Reward Sources:** For RLAIIF-style training, compare human-labeled vs AI-labeled preferences to confirm if synthetic labels can substitute (this is conceptual, as many uses have compared ChatGPT vs GPT-4 labels).
- **Dropout and Regularization:** Train with varying dropout rates or additional L2 regularization to test sensitivity to overfitting.

- **Training Length:** Run longer and shorter RL fine-tuning durations to find the point of diminishing returns or collapse. (In HRLAIF, going beyond ~500 steps hurt performance [36†L515-L523] .)

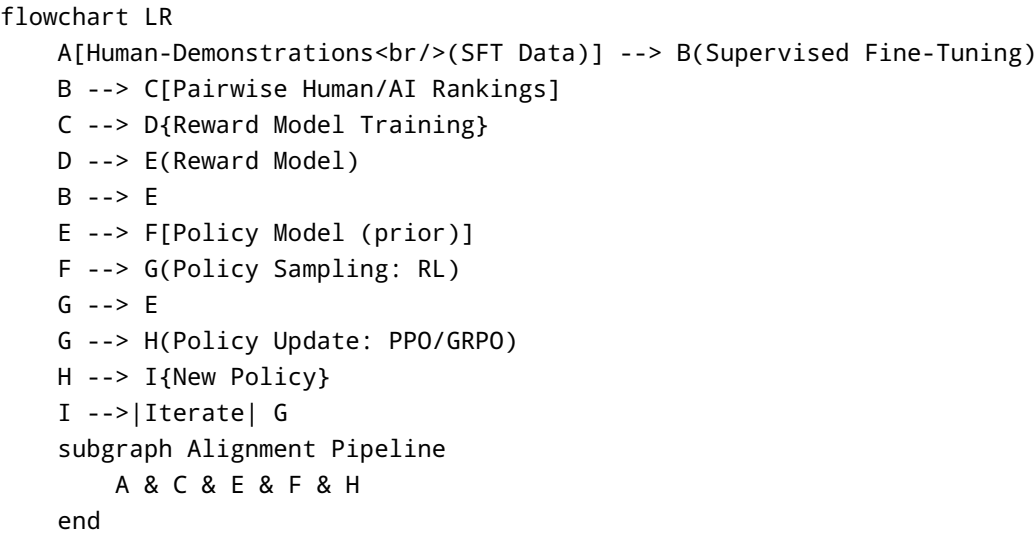


Figure: (Top) A generic alignment pipeline: first SFT on demonstrations, then train a reward model on preference data (human or AI labels), then perform policy optimization (PPO/GRPO) with the reward model in the loop [23†L2428-L2434] [51†L205-L213] . (Bottom) Gantt chart of phases: SFT, reward-model training, RL fine-tuning, and evaluation.

Sources: We drew hyperparameters and setups from primary papers and appendices: Ouyang *et al.* (2022) [23†L2428-L2434] [61†L569-L576] , Rafailov *et al.* (2023) [41†L1170-L1173] [56†L102-L106] , Peng *et al.* (2024, DeepSeekMath) [55†L1067-L1073] [51†L205-L213] , and Li *et al.* (2024, HRLAIF) [36†L448-L457] [36†L515-L523] . Where details were omitted in texts, we have noted assumptions or “unspecified.” All key values above are cited from these sources.